

Chapitre 4 : PHP et MySQL

(L'accès aux bases de données)

avec **PDO**

I. Introduction :

Problématique : Comment simplifier la gestion des bases de données avec PHP, car utiliser les fonctions basiques de PHP ne s'avère pas très pratique pour les gros projets.

PDO (PHP Data Objects) est là pour cela ! Cette classe objet implémentée directement dans PHP vous permet de manipuler des bases de données telles que MySQL, PostgreSQL, Oracle, Sybase, SQL Server ...

Intérêt:

- PDO répond à un grand besoin des développeurs qui travaillent sur des gros projets ou même des petits car les méthodes d'accès aux bases de données avec PHP sont très différentes d'un SGBD à un autre.
- PDO propose donc de réunir et d'unifier l'accès aux données et par la même occasion de remplacer toutes les extensions plus ou moins complètes par un socle commun.
- PDO (***PHP Data Objects***) vise à permettre la création d'un code comportant des accès aux BDD en faisant abstraction du moteur de SGBD utilisé.

C'est une interface qui, à ce titre, permet aux codeurs l'utilisant de travailler sans avoir à trop se poser de questions par la suite.

Avantages de PDO :

- *Interface pour SGBD* : plus besoin de s'occuper de savoir quelle SGBD est derrière (en théorie) ;
- *Orienté objet* : les objets PDO et PDOStatement peuvent être étendus, il est donc tout à fait possible de personnaliser et remodeler une partie du comportement initial ;
- *exception* : les objets de l'interface PDO utilisent des exceptions, il est donc tout à fait possible d'intégrer facilement un système de gestion des erreurs.

II. Connexion à une base en utilisant la classe PDO :

Pour établir une connexion il suffit d'instancier un Objet de type PDO. Le constructeur prend comme paramètre une chaîne de caractère qui contient tous les détails de la connexion :

- **host**: L'adresse du serveur ("localhost" pour un serveur local)
- **dbname**: Le nom de votre base de données
- **login**: Le login qui vous permet de vous connecter à MySQL (root par défaut sur votre machine)
- **password**: Votre mot de passe vous permettant de vous connecter à MySQL (il n'y a pas de mot de passe par défaut sur votre machine)

Exemple :

```
<?php
$PARAM_hote='localhost'; // le chemin vers le serveur
$PARAM_nom_bd='Nom_BD'; // le nom de votre base de données
$PARAM_utilisateur='root'; // nom d'utilisateur pour se
connecter $PARAM_mot_passe=''; // mot de passe pour se connecter
$connexion = new
PDO('mysql:host='.$PARAM_hote.';dbname='.$PARAM_nom_b
d, $PARAM_utilisateur, $PARAM_mot_passe);
?>
```

III. Gestion des erreurs de connexion :

Pour cela on peut utiliser la structure **Try catch**.

Exemple :

```
<?php
try
{
$connexion = new
PDO('mysql:host='.$PARAM_hote.';dbname='.$PARAM_nom_b
d, $PARAM_utilisateur, $PARAM_mot_passe);
}
catch(Exception $e)
{
echo 'Erreur : '.$e->getMessage(). '<br />';
echo 'N° : '.$e->getCode();
}
?>
```

IV. Fermeture de la connexion MySQL avec PDO

Pour fermer la connexion, on détruit l'objet:

```
<?php
```

```
$connexion = null;
```

```
?>
```

V. Les méthodes exec et query de la classe PDO

PDO fait la distinction entre deux familles de requêtes : ceci est un peu déroutant au début, mais finalement assez simple quand on en a compris le principe et l'intérêt. Comme vous le savez, il est possible sur une BDD de récupérer des informations, mais aussi d'effectuer des changements (qui peuvent prendre la forme d'ajout, suppression ou modification) :

- Si vous souhaitez **récupérer** une information (SELECT), vous devrez alors utiliser **'query'** ;
- Si c'est pour **apporter des changements** (INSERT, UPDATE, DELETE) à la BDD, vous devrez utiliser **'exec'**

V.1. La lecture des données avec PDO en PHP

Pour récupérer des enregistrements provenant de la base de données, il faut utiliser la méthode **PDO::query()**.

Dans l'exemple suivant, on veut récupérer les Id et les noms de tous les clients à partir de la table Client ayant la structure suivante (Id, Nom, Prenom, Adresse, Tel, CA):

```
$reponse = $connexion->query('SELECT Id, Nom FROM Client');
```

Après l'exécution de cette méthode, les enregistrements ne sont pas affichés, ils sont mis en mémoire. Il faut donc aller les chercher pour pouvoir les afficher. Il existe plusieurs solutions pour cela que l'on va voir par la suite :

a. **PDO::fetch()**

La méthode **fetch()** permet de récupérer un enregistrement parmi tous les enregistrements retournés par la requête.

Voici un exemple d'utilisation :

```
// $donnees1 contient le premier enregistrement  
$donnees1 = $reponse->fetch()  
// $donnees2 contient le second enregistrement  
$donnees2 = $reponse->fetch()  
// $donnees3 contient le troisième enregistrement  
$donnees3 = $reponse->fetch()  
// Et ainsi de suite
```

Pour parcourir simplement tous les enregistrements, on utilise une boucle afin de les traiter les uns après les autres. A chaque fois qu'on appelle **\$reponse->fetch()**, on passe à l'enregistrement suivant:

```
// On récupère tout le contenu de la table Client
$reponse = $connexion->query('SELECT Id, Nom FROM Client');
// On affiche le résultat
while ($donnees = $reponse->fetch())
{ //On affiche l'id et le nom du client en cours
echo $donnees ['Id'];
echo $donnees ['Nom'];
}
```

L'avantage de cette méthode, c'est qu'à un instant t, on ne lit qu'un seul résultat, du coup, la mémoire du système n'est pas occupée par les autres entrées. Quand on a fini d'interpréter un résultat, on passe au suivant qui va utiliser la mémoire utilisée par le précédent et ainsi de suite. La boucle s'arrête lorsqu'on a parcouru tous les enregistrements. L'inconvénient de cette méthode est que l'on ne peut pas exécuter d'autres requêtes sur la même connexion PDO pendant le traitement du résultat.

Le résultat retourné est sous la forme d'un tableau. En effet, on peut accéder au nom du client soit grâce à l'index « nom » qui est le nom de la colonne soit avec l'index « 1 » qui correspond au numéro de la colonne.

- **CloseCursor :**

CloseCursor provoque la fermeture du curseur d'analyse des résultats. Il faut faire appel à cette méthode à chaque fois que nous avons fini de traiter une requête. Cela permet d'éviter d'avoir des problèmes à la requête suivante.

```
// On récupère tout le contenu de la table Client
$reponse = $connexion->query('SELECT Id, Nom FROM Client');
// On affiche le résultat
while ($donnees = $reponse->fetch())
{
//On affiche l'id et le nom du client en cours
echo $donnees ['Id'];
echo $donnees ['Nom'];
}
$reponse->closeCursor();
```

b. Les modes de lecture de PDO en PHP

PDO propose différents modes afin de choisir la structure de données que l'on souhaite retourner. Nous allons voir dans ce billet, les modes les plus utilisés à savoir

PDO::FETCH_BOTH, PDO::FETCH_NUM, PDO::FETCH_ASSOC et PDO::FETCH_OBJ.

Pour changer de mode, on peut soit le passer en paramètre du fetch/fetchAll mode comme indiqué au niveau de l'exemple suivant:

Exemple :

```
// On récupère tout le contenu de la table Client
$reponse = $connexion->query('SELECT Id, Nom FROM Client');
// On affiche le résultat
while ($donnees = $reponse->fetch(PDO::FETCH_ASSOC))
{
    //On affiche l'id et le nom du client en cours
    echo $donnees['Id'];
    echo $donnees['Nom'];
}
$reponse->closeCursor();
```

Soit on utilise la méthode `setFetchMode()` fournie par PDO:

```
// On récupère tout le contenu de la table Client
$reponse = $connexion->query('SELECT Id, Nom FROM Client');
$reponse->setFetchMode(PDO::FETCH_ASSOC);
// On affiche le resultat
while ($donnees = $reponse->fetch())
{
    //On affiche l'id et le nom du client en cours
    echo $donnees['Id'];
    echo $donnees['Nom'];
}
$reponse->closeCursor();
```

• **PDO::FETCH_BOTH**

C'est l'attribut choisi par défaut. Donc si vous ne choisissez aucun mode, alors c'est comme-ci vous aviez choisi **PDO::FETCH_BOTH**.

La structure retournée (pour le `fetch()`) est:

Array

```
(
    [Id] => 1
    [0] => 1
    [Nom] => Nom0
    [1] => Nom0
)
```

Cette solution n'est pas la meilleure, en effet, car les données sont dupliquées.

• **PDO::FETCH_ASSOC**

Contrairement à **PDO::FETCH_BOTH**, **PDO::FETCH_ASSOC** va permettre de retourner un tableau associatif indexé seulement par le nom de la colonne. C'est la solution qu'il faut privilégier.

Array

```
(
    [Id] => 1
    [Nom] => Nom0
)
```

• PDO::FETCH_NUM

Contrairement à PDO::FETCH_ASSOC, PDO::FETCH_NUM va permettre de retourner un tableau associatif indexé par le numéro de la colonne.

Array

```
(  
    [0] => 1  
    [1] => Nom0  
)
```

Ce mode est le plus rapide mais le problème, c'est qu'il n'est pas pratique de travailler seulement avec le numéro des colonnes. Effet, si on supprime une colonne d'une table de la base de données, les résultats seront faussés.

• PDO::FETCH_OBJ

Ce mode permet de retourner un objet avec les noms de propriétés qui correspondent aux noms des colonnes.

StdClass Object

```
( [Id] => 1  
  [Nom] => Nom0  
)
```

V.2. Insertion, modification et suppression de données

La méthode exec() exécute une requête et retourne le nombre d'enregistrements affectés.

• INSERT

```
//On insère HAMZA Yosra dans la base de données  
//On en profite pour récupérer le nombre de lignes insérées (ici 1)
```

```
$nombreDeLigne = $connexion->exec("INSERT INTO Client (Nom, Prenom)  
VALUES ('HAMZA','Yosra')");
```

Lastinsertid

Dans certains cas, il peut être utile de connaître l'identifiant de l'élément que l'on vient d'insérer. Par exemple, on vient d'insérer HAMZA Yosra, pour obtenir son identifiant, on écrit la ligne suivante:

```
$id=$connexion->lastinsertid();
```

• UPDATE

```
//On modifie un enregistrement dans la base de données  
//On en profite pour récupérer le nombre de lignes modifiées (ici 1)
```

```
$nombreDeLigne = $connexion->exec("UPDATE Client SET Prenom = 'Alia' WHERE  
Nom = 'MAALOUL');"
```

• DELETE

```
//On supprime HAMZA Yosra dans la base de données  
//On en profite pour récupérer le nombre de lignes supprimées ici)
```

```
$nombreDeLigne = $bdd->exec("DELETE FROM Client  
WHERE Prenom = 'Yosra '  
AND Nom = 'HAMZA');
```

• Détection d'une erreur

Pour détecter une erreur lors de l'exécution de la requête, on vérifie la valeur de la variable retournée par la méthode **exec()**:

```
$res = $connexion->exec("DELETE FROM Client WHERE Id=2");  
if($res === FALSE)  
{  
    die('Erreur dans la requête')  
}  
else if($res === 0)  
{  
    echo "Aucune suppression effectuée";  
    echo "Ce client n'existe pas";  
}  
else  
{  
    echo "$res lignes ont été supprimées";  
}
```

On peut s'apercevoir que nous avons utilisé ici '===' et non '=='. En effet le '==' ne compare que la valeur et non le type. Or '0' et 'FALSE' ont la même valeur, du coup, le script ne ferait aucune différence entre les deux. Le '===' quant à lui vérifie l'égalité de valeur et de type. Ainsi 'FALSE' et '0' seront différenciés.

V.3. La méthode prepare

Cette méthode prépare une requête SQL à être exécutée en offrant la possibilité de mettre des marqueurs qui seront substitués lors de l'exécution.

Il existe deux types de marqueurs qui sont respectivement ? et les **marqueurs nominatifs**. Ces marqueurs ne sont pas mélangeables : donc pour une même requête, il faut choisir l'une ou l'autre des options.

Le principe des requêtes préparées est de créer un modèle de requête et de l'enregistrer sur notre SGBD. Le modèle est supprimé à chaque fermeture de la connexion avec la base de données. Lorsqu'on exécute une requête, la base de données va l'analyser, la compiler, l'optimiser puis l'exécuter. Le but des requêtes préparées est de ne pas répéter toutes ces actions lorsqu'on exécute des requêtes identiques.

Pour mieux comprendre, prenons un exemple. Je souhaite exécuter une requête d'insertion afin

d'ajouter le client « Alia MAALOUL » dans la base de données. La base de données va alors analyser, compiler, optimiser puis exécuter la requête. Grâce aux requêtes préparées, si avec la même connexion PDO je souhaite ajouter un autre client « Yosra HAMZA » (J'utilise la même requête d'insertion, seulement les paramètres sont modifiés, à savoir le nom et le prénom), alors la base de données va directement exécuter la requête car les étapes préliminaires (analyse, compilation et optimisation) ont déjà été effectuées précédemment lors de l'ajout de « Alia MAALOUL ».

Les avantages de ces requêtes, c'est qu'elles permettent de réduire le temps d'exécution lorsque nous souhaitons exécuter une même requête plusieurs fois.

De plus, les requêtes préparées sont automatiquement protégées contre les injections SQL. En revanche, si on exécute une seule fois une requête préparée, alors son temps d'exécution sera (très) légèrement plus long qu'une requête simple mais vous y gagnerez en sécurité.

V.4. Remplacer les paramètres

Pour construire un modèle de requête, il faut remplacer chaque paramètre par un paramètre précédé de « : » ou d'un point d'interrogation. C'est ceci qui nous permet de nous protéger des injections SQL.

```
$requete = "INSERT INTO Client (Nom, Prenom) VALUES (:unNom, :unPrenom)";
```

//ou

```
$requete = "INSERT INTO Client (Nom, Prenom) VALUES (?, ?)";
```

V.5. Préparer la requête

Il faut maintenant préparer la requête

```
$requete = "INSERT INTO Client (Nom, Prenom) VALUES (:unNom, :unPrenom)";
```

```
$stmt = $connexion->prepare($requete);
```

En cas d'échec, la méthode prepare retourne FALSE.

V.6. Associer les paramètres à des valeurs/variables

Il faut maintenant remplir les paramètres. Il existe deux solutions pour cela:

- **Stocker la valeur des paramètres dans un tableau passé en paramètre de la méthode execute()**

```
$requete = "INSERT INTO Client (Nom, Prenom) VALUES (:unNom, :unPrenom)";
```

```
$stmt = $connexion->prepare($requete);
```

```
$stmt->execute(array(
    'unNom' => 'MAALOUL',
    'unPrenom' => 'Alia'
));
```

- **Utiliser les Binds**

Le principe est de lier une variable ou une valeur à un paramètre sans passer par un tableau. **BindValue**

On associe le paramètre à une valeur:


```
$requete = "INSERT INTO Client (Nom, Prenom) VALUES (:unNom, :unPrenom)";
$stmt = $connexion->prepare($requete);
$stmt->bindValue('unNom', 'MAALOUL');
$stmt->bindValue('unPrenom', 'Alia');
$stmt->execute();
```

BindParam

On associe le paramètre à une variable. C'est à dire qu'après avoir lié le paramètre et la variable, on peut changer la valeur de la variable, le changement sera prit en compte.

```
$requete = "INSERT INTO Client (Nom, Prenom) VALUES (:unNom, :unPrenom)"; $stmt =
$connexion->prepare($requete);
$nom = 'MAALOUL';
$prenom = 'Alia';
$stmt->bindParam('unNom', $nom);
$stmt->bindParam('unPrenom', $prenom);
//On ajoute Alia MAALOUL
$stmt->execute();
//On modifie la valeur des variables
$nom='HAMZA';
$prenom='Yosra';
//On ajoute Yosra HAMZA
$stmt->execute();
```

Exemple d’affichage

```
<?php
// ouverture d'une connexion ...

$requete_prepare_1=$connexion->prepare("SELECT Id FROM client");
// on prépare notre requête

$requete_prepare_1->execute();
while($lignes=$requete_prepare_1->fetch(PDO::FETCH_OBJ
)) {
    echo $lignes->Id.'<br />';
}
?>
```